

06 · Multicore-Echtzeit

AMP, Shared Memory, Cache und MPU · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 55

Willkommen in der Multicore-Königsklasse!

- Sechster Termin: zwei Kerne, ein Chip, keine automatische Isolation
- Labor: **M7 Bare-Metal**, **M4 FreeRTOS** (CMSIS-RTOS2)
- HSEM kommt hier als **Boot-Handshake** vor, wenn das Template ihn erzeugt
- HSEM als Laufzeitsperre bleibt Termin 8
- Nächstes Mal: Tracing, nicht die volle HSEM-Vorlesung

Notizen

Keine .ioc und kein HAL-Baum im Kursrepo. Funktionsnamen, Takte und Boardrevision am generierten Projekt prüfen. NUCLEO-H755ZI-Q nur, wenn Cube dieses Board erzeugt hat.

Folie

2 / 55

Agenda für heute

1. Ein Kern, SMP, AMP
2. H755: Domänen, Bus, Boot
3. Zwei Images, ein SRAM4-Block
4. Cache, Ordnung, Ownership
5. MPU und Ping-Pong

Notizen

EDF mit $U \leq 1$ galt für einen Prozessor, $D = T$, ohne versteckte Kosten. Zwei Kerne übernehmen diese Formel nicht unverändert.

Recap: Die Grenzen des Single-Core

Didaktische Aufteilung, keine Messung:

Arbeit	ein Kern	Laboridee
Regelung	teilt sich die Zeit	eher M4
Netz, GUI	dieselbe CPU	eher M7

Gemeinsames SRAM, die Busbrücke und DMA bleiben Störquellen.

Vereinfacht, nur dynamische Schaltleistung:

$$P_{\text{dyn}} \approx \alpha C V^2 f$$

Spannung, Aktivität und Leckstrom kommen dazu. Keine gemessene Kurve.

Notizen

Frage: Welche Kosten bleiben mit einem zweiten Kern? Interferenz, Kopien, Protokoll, Boot. Ein rechnerisches $U = 1$ auf einem Kern hatte keine Reserve für IRQ und OS.

Erhöhung der Taktfrequenz?

- Höheres f erhöht P_{dyn} , nicht „exponentiell“ als einziges Modell
- 480 MHz (M7) und 240 MHz (M4) sind spezifizierte Maxima
- 400/200 MHz nur, wenn der Clock-Tree, VOS und die Flash-Wartezeit das hergeben
- Mehr Takt parallelisiert keine zwei Aufgaben auf einem Kern

Notizen

Die Laborfrequenz nicht aus dem Datenblatt-Maximum abschreiben.

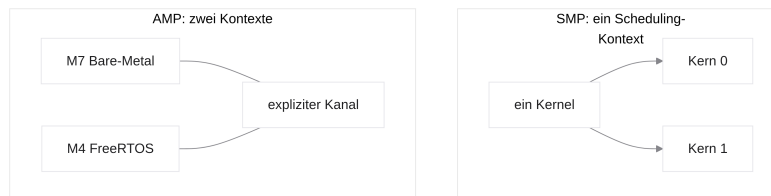
Der Ausweg: Multicore-Architekturen

- Zwei Kerne können rechnen, solange ihre Datenwege frei sind
- Partitionieren heißt: getrennte Firmware und klare Zuständigkeit
- Es heißt nicht: der andere Kern ist zeitlich unsichtbar

Notizen

AMP erleichtert die Trennung. Es ist kein Sicherheitsnachweis.

Multiprocessing-Architekturen



- SMP: gemeinsamer OS-Kontext, nicht „Kerne müssen identisch und gleichtaktig sein“
- Ready-Queues dürfen global oder je Kern sein, plus Affinität
- Migration verschiebt einen gesicherten Kontext; der Zielcache ist nicht leer versprochen

Notizen

Das rechte Bild ist das Labor. Links ist die Definition, nicht der H755-Zustand heute.

Symmetrisches Multiprocessing (SMP)

- Ein Scheduler sieht die Kerne
- Eine Task kann, wenn die Politik das erlaubt, den Kern wechseln
- Dabei werden Register gesichert, nicht der ganze Cache „genullt“
- Harte Fristen sind unter Annahmen möglich und unter anderen Annahmen schwer

Notizen

Titel der alten Folie „SMP scheitert oft“ wird zur Herausforderung, nicht zum Verbot.

Wie funktioniert ein SMP-Scheduler?

- Globale Queue: ein Kern nimmt die höchste bereite Aufgabe
- Oder: Warteschlange je Kern, der Scheduler schiebt nur bei Affinität oder Last
- Beides ist SMP, solange es **ein** Betriebssystemkontext ist
- Keine Pflicht zu genau einer Ready-Liste

Notizen

FreeRTOS auf nur dem M4 ist kein SMP über M7 und M4.

Warum SMP für harte Echtzeit oft scheitert

Besser: **Echtzeit-Herausforderungen bei SMP**

- Wer darf wohin migrieren?
- Geteilte Caches und Busse erzeugen Interferenz
- Sperren müssen kernübergreifend gedacht werden
- Das ist kein Beweis, dass SMP nie hart echtzeitfähig ist

Notizen

Geeignete Affinität, Budgets und ein Interferenzmodell können Fristen tragen. Die Analyse wird nur größer als auf einem Kern.

Asymmetrisches Multiprocessing (AMP)

- Zwei Firmware- und oft zwei OS-Kontexte
- Beispiel allgemein: Cortex-A mit Linux und Cortex-M mit RTOS
- Beispiel **dieser** Chip: M7 Bare-Metal und M4 FreeRTOS
- Der H755-M7 ist hier kein Linux-Standard

Notizen

Die beiden Beispiele nicht ineinanderzeichnen.

Software-Aufteilung in AMP

- M4: Tasks, Tick, Queues **dieses** Kernels
- M7: Superloop oder eigene Struktur, ohne diese Queue-IDs
- Eine lokale `osMessageQueueId_t` im Shared SRAM wird keine Inter-Core-Queue
- Der Kanal ist ein vereinbartes Speicherlayout plus Protokoll

Notizen

Frage: Warum darf die Queue-ID nicht auf dem anderen Kern liegen? Weil Handle, Heap und Listen dem einen Kernel gehören.

Wie kommunizieren AMP-Kerne?

Mittel	hier
Software-Puffer	unser SRAM4-Block
Hardware-Mailbox / FIFO	anderes Gerät, nicht dieser Ping-Pong
HSEM	Boot-Signal, später auch Sperre
lokale RTOS-Queue	nur M4-intern

Notizen

Shared Memory ist der Ort. Das Protokoll sagt, wer schreiben darf.

Zusammenfassung Block 1

- $U \leq 1$ bleibt ein Ein-Kern-Satz mit Annahmen
- SMP teilt einen Scheduler, AMP teilt ihn nicht
- M7-Rechnen ohne gemeinsame Zugriffe stört den M4 weniger als M7-Buslast
- „100 % M7-Last wirkt nie auf den M4“ ist falsch

Notizen

Nächster Block: wo die Last sich trifft.

Hardware Deep-Dive: Der STM32H755

Vereinfachte Karte, Details in RM0399 der verwendeten Revision:

Domäne	in dieser Zeichnung
D1	M7, TCM, AXI, Flash-Anbindung
D2	M4, viel AHB-Peripherie, Ethernet 10/100, USB
D3	SRAM4, langsame I/Os

Nicht jede schnelle Peripherie liegt in D1. Ethernet ist nicht Gigabit.

Notizen

Die Zeichnung ist bewusst grob. Wer ein Blockbild aus dem Referenzhandbuch abzeichnet, prüft die Revision.

Der Cortex-M7 (Das Muskelpaket)

- Spec bis 480 MHz, L1-I- und D-Cache, TCM
- TCM ist nicht der direkte Arbeitsplatz des M4
- Cache macht wiederholte Zugriffe schnell und fremde Werte unsichtbar
- Labor darf 400 MHz nur nach Clock-Tree-Prüfung sagen

Notizen

TCM in der Shared-Variable wäre für den M4 unerreichbar.

Der Cortex-M4 (Das Präzisionswerkzeug)

- Spec bis 240 MHz
- Kein M7-artiger L1-Daten-/Instruktionscache
- Der ART-Beschleuniger auf dem Flash-Pfad bleibt
- Wait-States, Bus, IRQs und die Speicherregion bleiben in der Zeit
- „Jeder Zugriff gleich viele Takte“ und „auf die Nanosekunde“ streichen

Notizen

Ohne L1-D-Cache ist der M4 nicht automatisch interferenzfrei.

Folie

17 / 55

Die Bus-Matrix (AHB / AXI)

- Parallele Wege, wenn die Pfade verschieden sind
- Dieselbe SRAM-Bank oder dieselbe Brücke serialisiert
- Ein Konflikt braucht nicht denselben CPU-Takt
- „AXI-SRAM immer 1 Takt“ ist keine universelle Garantie

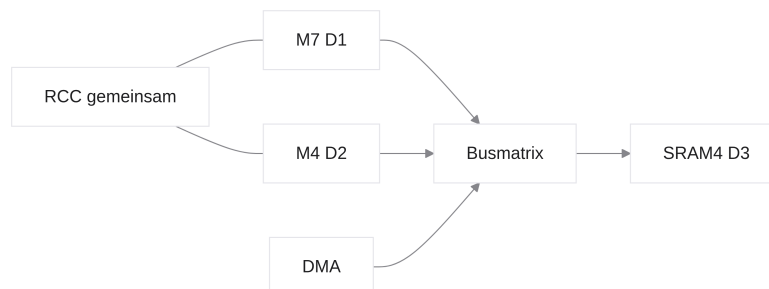
Notizen

Die Matrix beseitigt Interferenz nicht. Sie verschiebt sie auf die gemeinsamen Ziele.

Folie

18 / 55

Die 3 Power-Domänen (D1, D2, D3)



Oben getrennte Rechenkerne, unten gemeinsame Ziele.

Notizen

Eine reine Registerschleife im M7-TCM teilt sich den SRAM4-Port nicht. Sobald der M7 SRAM4, DMA oder RCC anfasst, kann der M4 warten. Qualitativ, ohne WCET-Zahl.

Taktversorgung (RCC)

- Ein Taktbaum, keine private PLL je Kern
- Wer `SystemClock_Config` zweimal ausführt, schreibt dieselben Register
- Im Labor initialisiert der M7 die gemeinsamen Takte, bevor der M4 weiterläuft
- Maxima 480/240 MHz zeichnerisch von der gewählten Laborfrequenz trennen

Notizen

GPIO-Ports teilen Register. Zwei Pins am selben Port sind kein konfliktfreier Read-Modify-Write.

Der Boot-Prozess: Wer weckt wen?

Übliches CubeMX-Muster, nicht „M4 bleibt in Reset“:

1. Option Bytes legen fest, wer überhaupt startet; Standard oft: beide Bootcodes
2. M4 richtet eine HSEM-Benachrichtigung ein und geht in **Stop**
3. M7 wartet auf diesen Zustand, setzt Takt und Shared-Block
4. M7 gibt das HSEM frei, M4 läuft weiter ins RTOS

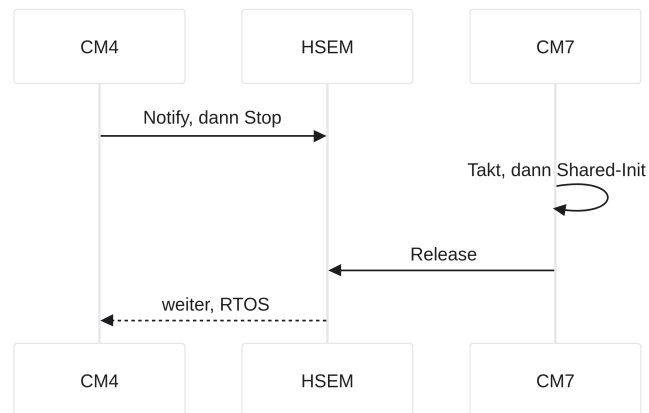
Stop, Reset und abgetakteter Kern sind drei Zustände.

Die konkrete HSEM-ID und die Funktionsnamen stehen in der generierten `main.HAL_PWREx_ReleaseCore` nicht aus dem Gedächtnis ergänzen.

Notizen

Begrenzte Wartezeit, sonst Diagnose-LED oder UART, keine Endlosschleife ohne Spur. Dieser HSEM-Schritt ist der Boot, nicht die Sperre für jedes Payload-Byte.

Hardware-Register: Den M4 wecken



Warten ohne Fortschritt: Diagnosekasten, nicht stiller Reset.

Notizen

Vorlage AN5361 und das eigene Template. Wenn das Projekt eine andere Option-Byte-Variante nutzt, gewinnt das Projekt.

Pin-Zuweisung im Dual-Core

- Pin Context steuert, **welcher Generator** den Init-Code schreibt
- Das ist keine Hardware-Sperre für den anderen Kern
- RCC und jeder Peripherieblock brauchen eine Zuständigkeit
- Gleicher GPIO-Port, verschiedener Pin: immer noch ein Register

Notizen

Beide Kerne dürfen nicht gleichzeitig GPIOE->MODER lesen-ändern-schreiben.

Toolchain und Projektstruktur

- Zwei ELF-Dateien, zwei Vektortabellen, zwei Linkerlayouts
- CM7 in Bank 1 und CM4 in Bank 2 ist eine mögliche Konvention
- Startadresse und Option Bytes müssen dazu passen
- Ab CubeIDE 2.0 oft: CubeMX separat, dann Import
- Eine Launch-Gruppe ist nicht automatisch da

Notizen

Keine erfundenen Menüscreenshots. Die Kursversion am Rechner nachsehen.

Debugging im AMP-System

- Eine ST-LINK- oder OpenOCD-Sitzung, nicht beides gemischt beschreiben
- Zwei Images laden, Reset-Reihenfolge zum Boot-Handshake passen
- „Halt all cores“ und Cross-Trigger können den Stop-Weckruf verfälschen
- Nach dem Halt die HSEM- und Stop-Zustände prüfen, nicht nur den PC

Notizen

Der genaue Klickpfad hängt von der IDE-Version ab. Offen, solange kein Projekt im Repo liegt.

Zusammenfassung Block 2

- D1/D2/D3 grob, Ethernet in D2, 10/100
- Bus parallel nur auf freien Wegen
- Boot: Stop und HSEM, nicht Reset-Leitung
- Zwei Images, eine Shared-Adresse

Notizen

Als Nächstes das Layout, das beide Linker meinen.

Die Speicherarchitektur (RAM)

Übliche H755-Anker, im Datenblatt der Revision prüfen:

Region	Adresse	Rolle im Labor
DTCM/ITCM	M7-lokal	nicht der Shared-Puffer
SRAM1–3	D2	M4-Heap/Stack, nicht automatisch shared
SRAM4	0x38000000–0x3800000F	64 KiB, gewählter Kanal
Backup-SRAM	andere Adresse	nicht SRAM4, nicht immer im Standby erhalten

SRAM4 ist die **Laborentscheidung**, keine Vorschrift für jede Anwendung.

Notizen

Bandbreite, DMA-Erreichbarkeit und Energiemodus können eine andere Bank nahelegen.

Wo legen wir den Shared Memory an?

- Beide Linker reservieren **dieselbe** Spanne und legen sonst nichts hinein
- Heap und Stack bleiben außerhalb
- NOLOAD heißt: der Loader schreibt die Sektion nicht
- Es heißt nicht: jedes Startup lässt die Bytes in Ruhe
- `.bss-Zero` und `.data-Copy` beider Images prüfen

Notizen

Nach Reset ist der Inhalt nicht als saubere Null voranzusetzen. M7 schreibt Zustand und Payload, bevor M4 kommuniziert.

Das Linker-Script (.ld)

Schema, keine abgelesene Map:

- Region SRAM4 einmal, nicht ein zweites RAM_D3 daneben
- Sektion .shared mit NOLOAD, KEEP, Alignment 32
- ASSERT auf Größe und Adresse
- Symbol sd in **beiden** Maps gleich, sonst ist es nicht derselbe Block

Notizen

Die echten .ld erzeugt Cube. Wir fügen eine Sektion ein und duplizieren die Speicherliste nicht. Map-Vergleich ist Pflicht und im Repo noch nicht ausführbar.

Eine eigene Sektion anlegen

```
.shared (NOLOAD) :  
{  
    . = ALIGN(32);  
    KEEP (*(.shared))  
    . = ALIGN(32);  
} >RAM_D3
```

Nur wenn RAM_D3 schon die SRAM4-Region ist. Sonst den vorhandenen Namen nehmen.

Notizen

KEEP gegen Garbage Collection. Alignment 32 wegen einer späteren Cache-Line, auch wenn die Basisregion non-cacheable ist.

Zuweisung im C-Code (GCC-Attribute)

```
/* shared_data.h: nur Typ und extern */
extern volatile ipc_block_t ipc
    __attribute__((section(".shared")));

/* genau eine .c je ELF */
volatile ipc_block_t ipc
    __attribute__((section(".shared")));
```

Das Attribut allein setzt nicht die Adresse. Linker, Alignment und weitere Objekte entscheiden.

Notizen

Dieselbe .c nicht in beide Projekte kopieren und zusätzlich im Header definieren. Einmal pro Image.

Das Schlüsselwort volatile

- `volatile` zwingt den Compiler zu den Zugriffen, die **dieser** Compiler dafür definiert
- Es umgeht nicht den D-Cache
- Es ist kein Mutex und kein atomares Read-Modify-Write
- Es veröffentlicht nicht „den ganzen Struct auf einmal“

Notizen

GCC-Volatile-Doku. Für die Reihenfolge kommt eine Barriere dazu, für den Cache die MPU oder eine Wartung.

Reicht volatile für Multicore?

Vier verschiedene Probleme:

Problem	Mittel	löst allein nicht
Compiler hält einen Wert	<code>volatile</code>	Cache, Reihenfolge, Konflikt
veraltete Cachezeile	MPU oder Clean/Invalidate	wer schreiben darf
Reihenfolge der Zugriffe	DMB / DSB	Cache-Inhalt
beide schreiben	Ownership oder HSEM	Sichtbarkeit

DMB ordnet. DSB wartet auf den Abschluss. Keines reinigt den D-Cache.
Keine Armv8-M-Acquire-Intrinsics ungeprüft auf M4/M7.

Notizen

Frage zur Brücke: Warum reicht non-cacheable nicht? Weil die Reihenfolge Payload vor Zustand und das alleinige Schreibrecht bleiben.

Struktur der Ping-Pong Daten

Altes Bild mit zwei `uint8_t` und einem `uint32_t`: natürliche Ausrichtung **8 Byte**, Offsets 0, 1 und 4, nicht 12.

Gewählter Block:

```
typedef struct {
    uint32_t magic;
    uint32_t state;    /* TURN_M7, REQUEST_M4, RESPONSE_M7 */
    uint32_t payload;
} ipc_block_t;
```

`_Static_assert` auf Größe und Alignment im Zielbuild. Kein `packed` als Multicore-Trick.

Nur der Berechtigte schreibt `payload`. Der Zustand wechselt danach.

Notizen

M7 initialisiert magic, state und payload, solange M4 noch nicht kommuniziert. NOLOAD-Müll ist kein Startwert 0.

Folie

34 / 55

Zusammenfassung: Shared Memory

- Eine Adresse, zwei ELF, eine Definition je Image
- M7 initialisiert, M4 löscht den Block beim Start nicht
- Ein Core-Reset mitten im Ping-Pong ist nicht das Basislabor
- Ein gemeinsamer Reset muss den Handshake neu durchlaufen

Notizen

Was steht nach dem Reset vor der Init? undefiniert, bis M7 schreibt.

Folie

35 / 55

Was ist ein CPU-Cache?

- Zeile typisch 32 Byte auf diesem M7
- Ein Lese-Treffer kann einen alten Wert liefern
- Ein Schreib-Treffer kann nur die Zeile ändern
- Der M4 sieht SRAM, nicht die Dirty-Line des M7

Notizen

Auch Lesen ist ein Kohärenzproblem, nicht nur Schreiben.

Folie

36 / 55

Write-Through vs. Write-Back

- Write-Through: Schreiben geht Richtung Speicher, hilft nicht für die Richtung M4 → M7
- Write-Back: Rückschreiben bei Verdrängung oder Clean, nicht nur „wenn der Cache voll ist“
- Clean schreibt Dirty-Daten
- Invalidate verwirft die **Kopie**, nicht benachbarte SRAM-Bytes
- Verloren gehen Dirty-Daten, die noch nicht zurückgeschrieben waren

Notizen

Eine alte Line über neue M4-Daten zu cleanen überschreibt die Antwort. Deshalb nicht „ganzes Struct immer clean+invalidate“.

Folie

37 / 55

Das Drama der Cache-Kohärenz

Zwei Zustände, dieselbe Zeile:

1. M7 hat den neuen Request nur dirty. M4 liest den alten SRAM-Wert.
2. M4 hat die Antwort ins SRAM geschrieben. M7 liest noch seine alte Kopie.

Notizen

Werte und Pfeile getrennt halten. Keine Zeitachse, solange wir keine Latenzen messen.

Folie

38 / 55

Visualisierung: Cache Coherency Failure



Notizen

Synthetisches Bild. Die Laborbasis umgeht den D-Cache per MPU, statt jede Richtung von Hand zu warten.

HW-Lösung in PCs: Cache-Snooping

- Kohärenz kann Cache-zu-Cache liefern, nicht nur über RAM
- MESI ist ein Lehrbild, nicht das vollständige Modell jedes PCs
- Daraus folgt nicht, dass PC-Kerne nie hart echtzeitfähig sind
- Der H755 hat diese Kohärenz zwischen M7-L1 und M4 nicht

Notizen

Konzeptfolie. Keine Register des M7 daraus ableiten.

Software-Lösung 1: Manuelles Management

Option, nicht die Basis:

- Getrennte 32-Byte-Blöcke, ein Schreiber
- Clean nur der eigenen neuen Daten, bevor der andere lesen muss
- Invalidate nur, bevor man fremde neue Daten lesen will, und ohne dirty Nachbarschaft zu opfern

Eine Line mit Payload, Flag und fremder Dirty-Variable ist der Fehlerfall.

Notizen

Alignment der Startadresse genügt nicht. Größe und wer sonst in der Line schreibt, zählen.

Nachteile der Software-Lösung

- Leicht die Richtung zu verwechseln
- Clean der veralteten Line kann M4-Daten überschreiben
- Invalidate kann eigene Dirty-Bytes verwerfen
- Nur aufnehmen, wenn der Code wirklich geprüft ist
- Das Basislabor macht die Region non-cacheable

Notizen

Zwischen den Varianten neu starten. Attribute nicht umschalten, während beide schreiben.

Hardware-Lösung 2 (Best Practice): Die MPU

Für SRAM4 im Basislabor, Attribute aus AN4838, Konstanten im HAL-Header prüfen:

Feld	Wert
Basis / Größe	0x38000000, 64 KiB
Typ	Normal, TEX = 001, C = 0, B = 0
Rechte	Schreiben erlaubt, Execute Never
Shareability	passend zur Doku der Region setzen

Reihenfolge: MPU, dann D-Cache an, dann Shared-Zugriff.

Der Pfeil umgeht den D-Cache. Ownership und DMB bleiben.

Notizen

Überlappende Regionen: die höhere Regionsnummer gewinnt auf dem M7. Eine cacheable Region darüber würde die Absicht kaputt machen. Das im MPU-Register nachsehen.

Was bewirkt die MPU?

- Diese Region wird für den M7 nicht als cachebarer Normal-Speicher behandelt
- Sie macht die Bytes nicht „100 % sicher“
- Zwei Kerne ohne Cache brauchen das Protokoll trotzdem
- Weglassen der MPU erzeugt nur dann den Cachefehler, wenn der D-Cache an ist und der Weg cachebar bleibt

Notizen

CCR und die Regionsregister sind der Nachweis, nicht die Hoffnung. Eine eigene Write-Back-Variante nur als Versuch 2.

Laborvorbereitung: Der Ping-Pong Datenaustausch

Zustand: `TURN_M7`, `REQUEST_M4`, `RESPONSE_M7`.

M7 setzt vor der Freigabe Payload 0 und den ersten Request.

M4 macht daraus 1. M7 druckt 1, legt 2 hin, requested wieder.

M4 antwortet 3. Sichtbar: **1, 3, 5, ...** nicht zuerst 0.

Nach der Übergabe schreibt niemand weiter am Payload.

Notizen

Die 500-ms-Pause und die UART-Zeit sind die sichtbare Rate, nicht die IPC-Latenz. `osDelay(1)` ist ein Tick, nicht automatisch 1 ms.

Schritt 1: CubeMX Dual-Core Setup

- Board nur übernehmen, wenn es NUCLEO-H755ZI-Q ist
- Beide Kerne, FreeRTOS nur auf dem M4, CMSIS_V2
- UART-Retargeting einem Kern zuordnen
- Takt aus dem Cube-Tree abschreiben, nicht aus dieser Folie

Notizen

Stromversorgung und VOS bestimmen, ob 400/480 MHz legal sind. Offen ohne Projekt.

Schritt 2: Den Cortex-M4 wecken

Den generierten HSEM/Stop-Code behalten und prüfen:

- HSEM-Takt an
- ID notieren
- M4: Notify, Stop, begrenztes Warten auf M7-Seite
- M7: Shared-Init **danach** erst Release
- Fehler: Status ausgeben, nicht den Block löschen, während der andere läuft

Notizen

Keine zweite Wake-Funktion neben dem Template erfinden.

Schritt 3: Shared Memory Struct anlegen

Header gemeinsam, Definition je eine .c pro Image.

```
_Static_assert(sizeof(ipc_block_t) % 4 == 0).
```

Magic ungleich 0, damit M4 uninitialisierten Speicher erkennt und wartet oder den Timeout nimmt.

Notizen

M4 darf `memset` auf den Block nicht im Task-Start ausführen.

Schritt 4: Das Linker-Script anpassen

Beide Skripte: dieselbe Sektion, kein doppeltes RAM.

Map: `ipc` bzw. `sd` gleiche Adresse.

Startup: der Bereich darf nicht in der Zero-Tabelle liegen, sonst löscht ein Kern den laufenden Dialog.

Notizen

Schema-Snippet, keine behauptete Map aus diesem Repo.

Schritt 5: Die Ping-Pong Logik (Cortex-M4)

```
void IpcTask(void *argument) {
    (void)argument;
    for (;;) {
        if (ipc.state != REQUEST_M4) {
            osDelay(1); /* ein Tick, Frequenz aus der Config */
            continue;
        }
        __DMB();
        uint32_t v = ipc.payload + 1u;
        ipc.payload = v;
        __DMB();
        ipc.state = RESPONSE_M7;
        osDelay(1);
    }
}
```

Timeout über einen Zähler der Runden, nicht über HAL_GetTick des anderen Kerns.

Notizen

Während osDelay laufen andere M4-Tasks. Ein hängender Handshake ist kein kompletter CPU-Stillstand.

Schritt 6: Die Ping-Pong Logik (Cortex-M7)

```
ipc.payload = 0;
__DMB();
ipc.state = REQUEST_M4;
/* nach Antwort: */
__DMB();
log_u32(ipc.payload); /* 1, dann 3, dann 5 */
ipc.payload += 1u;
__DMB();
ipc.state = REQUEST_M4;
```

HAL_Delay(500) nur als sichtbare Pause. Nicht als Latenzmessung.

Keine Antwort innerhalb weniger Pausen: Fehlerzeile, Zustand stehen lassen, Schleife beenden.

Notizen

M7-HAL_Delay ist Busy-Wait auf dem M7. Der M4 kann trotzdem im RTOS weiterlaufen, solange der Bus das erlaubt.

Schritt 7: MPU im M7 konfigurieren

Vor SCB_EnableDCache() und vor dem ersten Shared-Zugriff.

Konstantennamen aus stm32h7xx_hal_cortex.h der installierten CubeH7-Version.

Nicht raten, wenn der Header abweicht.

Versuch 2, eigener Build, Neustart: Region cachebar, Write-Back, keine Wartung. Fehler diagnostizieren, nicht als garantierten Freeze verkaufen.

Notizen

Ohne aktiven D-Cache zeigt Versuch 2 nichts. CCR lesen.

Flashen & Debuggen im Labor

- Beide ELF laden, Reset so, dass der HSEM-Handshake durchläuft
- Erste drei Runden: M4 schreibt 1, 3, 5; M7 druckt sie
- UART-Kern ist der, der `printf` wirklich bedient
- Breakpoints auf beiden Kernen verschieben Timeouts

Notizen

ST-LINK-Ablauf der Kursversion. OpenOCD nicht in denselben Satz mischen, solange er nicht der verwendete Server ist.

Mutwilliger Fehler: MPU deaktivieren

Symptom	Prüfung	typische Ursache
M4 startet nicht	Stop, HSEM-ID	Boot, nicht Cache
falsche Adresse	beide Maps	Linker
Magic falsch	Speicherfenster	Init fehlt oder Zero läuft
M7 sieht alten Wert	CCR, MPU-Region	Cachezeile
Timeout	Zustand eingefroren	Ownership oder Barriere
falsches Image	ELF-Namen	Bank/Startadresse

GPIO oder UART-Status je Symptom, keine erfundene Messung.

Notizen

Ein blockierter M4-Task im `osDelay` lässt andere Tasks laufen. Das ist kein Beweis, dass der Kern steht.

Die Gefahr für nächste Woche (HSEM)

- Boot-HSEM ist heute erledigt
- Eine HSEM-Sperre für jeden Zugriff ist Termin 8
- HSEM ersetzt weder MPU noch DMB
- Ein lokaler FreeRTOS-Mutex sieht den M7 nicht

Notizen

Kursplan: Termin 7 Tracing, Termin 8 HSEM und Safety. „Nächste Woche HSEM“ wäre der falsche Verweis.

Zusammenfassung des Tages

- AMP partitioniert Firmware, nicht automatisch die Zeit
- Boot: M4 Stop, M7 init, HSEM Release
- Shared: eine Adresse, M7 initialisiert, Zustand vor Payload-Leser
- Cache: beide Richtungen; MPU non-cacheable plus DMB plus ein Schreiber
- Ausgabe 1, 3, 5 ist das Protokoll, nicht eine Messung

Notizen

Quellen: DS12919, RM0399 der Projekt-Revision, AN5361, AN5617, AN4839, AN4838, CMSIS-Cache und Intrinsics, GCC volatile, GNU ld NOLOAD. Offene Punkte: installierte Cube-Version, echte Map, UART-Pin, VOS.